- Eventually the unit tests will focus on as small a part of the code as you can manage. Having one test that makes sure all those little pieces correctly pass control between them prevents bugs from living in the gaps between the pieces.

Right now this test will fail—spectacularly. Absolutely none of the component bits are in place. So let's take this tiny step by tiny step, in each case minimally clearing the current error.

Pending Tests

In case it bothers you to see the integration test continue to fail while you write the unit tests that will make it pass, RSpec allows you to specify a test as pending or to skip it altogether. In RSpec, any it method defined without a block is considered to be "pending."

```
it "bends steel in its bare hands"
```

You can temporarily mark an it or describe block as pending by adding :pending as a second argument after the string:

```
it "bends steel in its bare hands", :pending do
  #anything
end
```

Alternatively, you can use the method pending in the spec:

```
it "bends steel in its bare hands" do
  pending "not implemented yet"
end
```

In RSpec all pending specs are actually run if there is code in the block part of the spec. The code is executed, with any failure in the pending spec treated as a pending result rather than a failure result. However, if the code in the pending spec passes, you'll get an error that effectively means, "You said this was pending, but lo and behold, it works. Maybe it's not actually pending anymore; please remove the pending status."

If you want the spec to not run, which means that you do not check for whether the spec passes, employ the preceding syntax but use skip instead of pending. Alternatively, you can prefix the method name with x, as in xit or xdescribe. A skipped test will not run, meaning you won't get any notification if the test suddenly starts to pass.